

Memory-Based Navigation Robot Using Encoder Feedback

Shaik Nazeer

Department of Electronics and Communication Engineering
Rajiv Gandhi University of Knowledge Technologies, Ongole
Email: shaiknazeer1428@email.com

Abstract—This paper discusses the design and implementation of a memory-based navigation robot that can record and reproduce a defined path through the use of encoder feedback. The system is designed to use an ESP32 microcontroller that can handle Bluetooth communication, motor control, and processing.

During manual navigation, the encoder pulses that correspond to the rotation of the wheels are constantly recorded and stored. The stored encoder pulses can then be used to reproduce the path autonomously without the need to use other navigation aids such as GPS. The proposed system can be used as an efficient solution for indoor navigation because GPS cannot be used indoors. The results show that the robot can reproduce the path successfully.

Index Terms—ESP32, Encoder Feedback, Autonomous Robot, Path Memory, Indoor Navigation

I. INTRODUCTION

Autonomous mobile robots have gained significant importance in recent years due to their wide range of applications in industries, healthcare, logistics, and service sectors. These robots are designed to operate with minimal human intervention, performing tasks such as material handling, inspection, surveillance, and delivery. Among the various functionalities of an autonomous robot, navigation plays a crucial role in determining its efficiency and reliability.

Navigation systems enable robots to move from one location to another while avoiding obstacles and maintaining accuracy. Traditional navigation techniques include GPS-based systems, LiDAR-based mapping, and vision-based approaches. GPS is widely used for outdoor navigation; however, it becomes ineffective in indoor environments due to weak or unavailable signals. On the other hand, LiDAR and vision-based systems provide accurate navigation but require expensive hardware and complex computational algorithms.

For indoor applications, where environments are structured and paths are often repetitive, the use of advanced navigation systems may not be necessary. The integration of multiple sensors and high-end processing units increases both the cost and complexity of the system. This creates a need for a simpler, cost-effective, and efficient navigation method that can operate reliably in indoor conditions.

In this context, the concept of memory-based navigation offers a practical solution. Instead of relying on external sensors or mapping techniques, the robot learns a path during manual operation and stores the movement data. This stored information can later be used to reproduce the same path

autonomously. Such an approach is particularly useful for tasks that involve repetitive movement along predefined routes.

The proposed system utilizes an ESP32 microcontroller along with encoder feedback to implement memory-based navigation. During manual control, the robot records encoder data corresponding to wheel movements. This data is then processed and stored in memory, forming a representation of the path taken. In autonomous mode, the robot uses the stored data to retrace the path without requiring continuous user input or external navigation aids.

The primary objective of this project is to design and develop a robot capable of recording and replaying a path using encoder feedback. The system aims to provide a low-cost and efficient alternative to conventional navigation methods, making it suitable for indoor applications and educational purposes.

Overall, this work demonstrates a simple yet effective approach to autonomous navigation, emphasizing affordability, ease of implementation, and reliability in structured environments.

II. PROBLEM STATEMENT

Indoor navigation of mobile robots remains a significant challenge due to the limitations of existing navigation technologies. Conventional systems such as GPS-based navigation are ineffective in indoor environments because GPS signals are weak or unavailable inside buildings. This restricts their usability in applications such as warehouses, hospitals, offices, and other enclosed spaces.

Advanced navigation techniques such as vision-based systems and SLAM (Simultaneous Localization and Mapping) provide accurate and intelligent navigation capabilities. However, these systems require high computational power, complex algorithms, and expensive sensors such as cameras and LiDAR. As a result, they significantly increase the overall cost and complexity of the system, making them unsuitable for low-cost and small-scale applications.

In many practical scenarios, robots are required to perform repetitive tasks along predefined paths rather than navigate dynamically. For such applications, the use of complex navigation systems is unnecessary and inefficient. There is a need for a simple, cost-effective, and reliable navigation method that can operate efficiently in indoor environments without

relying on expensive hardware or computationally intensive algorithms.

The challenge lies in developing a system that can accurately record a path during manual operation and reproduce the same path autonomously with minimal error. The system should be capable of utilizing basic hardware components while maintaining acceptable levels of accuracy and repeatability.

This work aims to address these challenges by proposing a memory-based navigation robot that uses encoder feedback for path recording and reproduction. By leveraging encoder data, the system captures movement information in terms of distance and direction, enabling the robot to retrace its path without the need for external navigation aids.

The proposed solution focuses on simplicity, affordability, and efficiency, making it suitable for educational purposes, small-scale automation, and structured indoor environments where predefined paths are commonly used.

III. RELATED WORK

A wide range of navigation techniques have been developed in the field of mobile robotics, each addressing different requirements such as accuracy, cost, and environmental adaptability. These techniques vary from simple sensor-based approaches to complex mapping and localization algorithms.

GPS-based navigation is one of the most commonly used methods for outdoor robotic applications. It provides global positioning information and enables robots to navigate over large distances. However, GPS signals are significantly weakened or completely unavailable in indoor environments, making this approach unsuitable for indoor navigation tasks.

Vision-based navigation systems utilize cameras along with image processing and computer vision algorithms to understand and interpret the surrounding environment. These systems can detect obstacles, recognize objects, and perform path planning. While they are highly effective in dynamic environments, they require high computational resources and are sensitive to lighting conditions, which increases system complexity and cost.

SLAM (Simultaneous Localization and Mapping) is an advanced navigation technique that allows robots to build a map of an unknown environment while simultaneously determining their position within it. SLAM-based systems often use sensors such as LiDAR, depth cameras, or ultrasonic sensors to achieve high accuracy. Despite their effectiveness, these systems involve complex algorithms and expensive hardware, making them less suitable for low-cost and small-scale applications.

In addition to these methods, other techniques such as line-following robots and sensor-based navigation using infrared or ultrasonic sensors have also been explored. While these methods are relatively simple and cost-effective, they are limited in flexibility and typically require predefined infrastructure such as tracks or markers.

Compared to the above approaches, memory-based navigation offers a simpler and more economical alternative.

Instead of relying on external sensors or real-time environment mapping, the robot records its movement path using encoder feedback during manual operation. The stored path data is then used to reproduce the same trajectory autonomously.

This approach is particularly suitable for applications where the navigation path is repetitive and predefined, such as in warehouses, industrial environments, and indoor delivery systems. Although it lacks adaptability to dynamic environments, its low cost, ease of implementation, and minimal hardware requirements make it an attractive solution for many practical scenarios.

Thus, the proposed system builds upon the concept of memory-based navigation, focusing on improving efficiency and reliability using encoder feedback and embedded control techniques.

IV. HARDWARE DESCRIPTION

This section describes the hardware components used in the development of the proposed memory-based navigation robot and the rationale behind their selection.

A. Hardware Implementation

The system is implemented using an embedded hardware architecture consisting of a microcontroller, motor driver, DC motors with encoders, and a power supply unit. The ESP32 microcontroller acts as the central processing unit, interfacing with all other components.

The motor driver module is used to control the direction and speed of the motors based on control signals from the microcontroller. The motors are equipped with encoders that generate pulses corresponding to wheel rotation, enabling feedback-based motion tracking.

All components are interconnected using appropriate wiring, ensuring proper signal flow and power distribution. The system is designed to be compact, efficient, and suitable for real-time operation.

B. Selection of Controller

The ESP32 microcontroller is selected due to its advanced features, including dual-core processing, built-in Bluetooth and Wi-Fi capabilities, and sufficient GPIO pins for interfacing with external devices.

Compared to traditional microcontrollers such as Arduino Uno, the ESP32 offers higher processing speed and better communication capabilities, making it suitable for real-time control applications. Its integrated Bluetooth module eliminates the need for additional communication hardware, reducing system complexity and cost.

C. Other Components Used

The following components are used in the system:

L298N Motor Driver: The L298N is a dual H-bridge motor driver capable of controlling two DC motors independently. It allows bidirectional control and speed regulation using PWM signals.

N20 DC Motors with Encoder: These motors provide the necessary mechanical motion. The attached encoders generate

pulses that are used to measure rotation and calculate distance traveled.

Wheel Encoders: Encoders convert mechanical motion into electrical signals, providing feedback for accurate motion control and path tracking.

12V Battery: The system is powered by a 12V battery, which supplies sufficient energy for motor operation and electronic components. Voltage regulation is used where necessary to ensure safe operation.

Chassis and Wheels: A sturdy robot chassis is used to mount all components securely. The wheels are selected to provide good traction and stability during movement.

Connecting Wires and Power Modules: Proper wiring and power distribution components are used to ensure reliable connections and prevent voltage fluctuations.

V. SYSTEM ARCHITECTURE

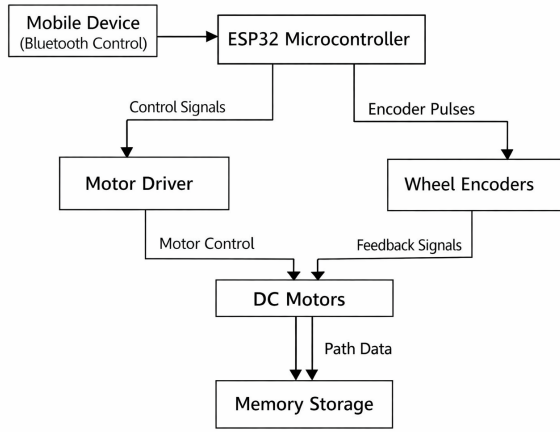


Fig. 1. Block diagram of the proposed system

The proposed system is designed as an embedded control system consisting of four major modules: the control unit, communication system, motor driver module, and feedback system. These modules work together to achieve precise and efficient motion control.

The **control unit** is implemented using the ESP32 microcontroller, which acts as the brain of the system. It is responsible for processing incoming commands, executing control logic, and managing communication between different modules. The ESP32 continuously monitors the system state and ensures proper coordination between input and output operations.

The **communication system** enables wireless interaction with the system using Bluetooth technology. Commands such as forward, backward, left, right, and stop are transmitted from a mobile device to the ESP32. This allows real-time control without physical connections, improving user convenience and flexibility.

The **motor driver module** consists of the L298N motor driver, which acts as an interface between the low-power control signals from the ESP32 and the high-power requirements

of the motors. It controls both the direction and speed of the motors using PWM (Pulse Width Modulation) signals.

The **feedback system** uses wheel encoders to provide real-time information about motor rotation. The encoder generates pulses corresponding to the movement of the wheels, which are processed by the ESP32 to determine speed, distance, and direction. This feedback mechanism ensures accurate motion control and helps in correcting errors during operation.

Overall, the integration of these modules results in a reliable and efficient system capable of precise motion control with real-time feedback.

VI. SYSTEM DESIGN

The overall system is designed using a modular approach to simplify development, testing, and scalability. Each module operates independently while maintaining synchronization with the central control unit, ensuring reliable and efficient operation.

The system is divided into multiple functional layers, including the power management layer, control layer, communication layer, and feedback layer. This layered architecture improves maintainability and allows individual modules to be modified or upgraded without affecting the entire system.

The hardware design consists of power distribution, signal processing, and control layers. The ESP32 microcontroller forms the core processing unit, interfacing with the motor driver and encoder modules through its GPIO pins. Proper voltage regulation is implemented to ensure that sensitive components such as the microcontroller receive stable voltage levels, while the motors are supplied with higher power from the battery.

The motor driver acts as an intermediary between the control unit and the motors, amplifying low-power control signals into high-current signals required for motor operation. PWM (Pulse Width Modulation) signals are used to control motor speed, enabling smooth acceleration and deceleration.

The software design follows an event-driven architecture, where the system continuously monitors for incoming Bluetooth commands. Upon receiving a command, the control logic determines the appropriate motor action and executes it in real time. This approach ensures minimal delay and efficient handling of user inputs.

Interrupt-based programming is used for encoder signal acquisition. Encoder pulses are captured using hardware interrupts, allowing precise counting without interrupting the main execution loop. This ensures accurate measurement of wheel rotation even at higher speeds.

The system also incorporates structured data storage mechanisms for recording path information. Encoder counts and movement directions are stored sequentially in memory, enabling efficient retrieval during path replay.

To enhance system reliability, special care has been taken to ensure proper isolation between high-power motor circuits and low-power control circuits. This minimizes electrical noise, prevents voltage fluctuations, and protects sensitive components from potential damage.

Additionally, grounding techniques and proper wiring practices are followed to reduce interference and improve signal integrity. The overall design ensures robustness, efficiency, and suitability for real-time embedded applications.

The modular and scalable nature of the system makes it easy to integrate additional features such as obstacle detection, advanced control algorithms, and IoT-based monitoring in future enhancements.

VII. HARDWARE COMPONENTS

The system is implemented using the following hardware components:

ESP32 Microcontroller: The ESP32 is a powerful and cost-effective microcontroller with built-in Wi-Fi and Bluetooth capabilities. In this system, it functions as the central processing unit, handling input commands, executing control algorithms, and generating output signals. Its high processing speed and multitasking capability make it suitable for real-time embedded applications.

The ESP32 acts as the central processing unit...

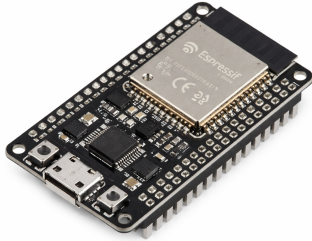


Fig. 2. ESP32 microcontroller

L298N Motor Driver: This module controls motor speed and direction...

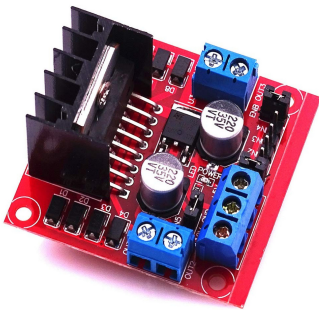


Fig. 3. L298N motor driver

The L298N is a dual H-bridge motor driver module used to control two DC motors simultaneously. It receives low-voltage control signals from the ESP32 and converts them into high-current signals required by the motors. It also allows bidirectional control, enabling the motors to rotate in both clockwise and counterclockwise directions.

N20 Motors with Encoder: N20 motors are compact DC geared motors widely used in robotics applications. These motors are equipped with encoders that generate pulses during rotation. The encoder data is used to measure speed and position, enabling precise motion control.

Wheel Encoders: Wheel encoders play a critical role in the feedback system. They convert mechanical motion into electrical signals in the form of pulses. By counting these pulses, the system can determine parameters such as distance traveled, rotational speed, and direction of movement.

12V Battery: Provides power...



Fig. 4. 12V battery

A 12V battery is used as the primary power source for the system. It provides sufficient energy to drive the motors and other electronic components. Voltage regulation may be used to supply appropriate voltage levels to sensitive components like the ESP32.

VIII. CONTROL STRATEGY

The robot operates using a differential drive mechanism, in which two independently driven wheels control the overall motion of the system. By varying the speed and direction of the left and right motors, the robot can perform various movements such as forward motion, backward motion, turning, and rotation in place.

The control strategy is primarily based on command-driven motion combined with encoder-based feedback. Commands are transmitted wirelessly via Bluetooth from a user interface, such as a mobile application. Each received command corresponds to a predefined motor action, which is executed by the ESP32 microcontroller through the motor driver module.

During manual operation, the system follows an open-loop control approach, where motor actions are directly based on user commands without continuous feedback correction. This mode is primarily used for recording the path, where encoder data is collected and stored for later use.

In the path reproduction phase, the system adopts a semi-closed-loop control strategy. The stored encoder values are used as reference inputs, and the robot attempts to replicate the same motion by generating corresponding motor control signals. Although encoder feedback is used to monitor movement, real-time correction is limited, which may lead to small deviations over longer distances.

Pulse Width Modulation (PWM) is employed to regulate motor speed. By adjusting the duty cycle of the PWM signals,

the effective voltage supplied to the motors is controlled, allowing smooth variation in speed. This enables better handling of acceleration and deceleration, reducing mechanical stress and improving motion stability.

The system also incorporates directional control logic, where specific combinations of motor inputs are used to achieve desired movements. For example, both motors rotating in the same direction result in linear motion, while opposite directions enable turning or rotation.

Synchronization between the two motors is an important aspect of the control strategy. Minor differences in motor characteristics or load conditions can lead to uneven motion. Encoder feedback helps in identifying such discrepancies, although full correction is not implemented in the current design.

One of the key limitations of the current control strategy is the absence of continuous feedback correction, which can result in cumulative errors. To address this, future improvements may include the implementation of closed-loop control techniques such as Proportional-Integral-Derivative (PID) control. A PID controller can dynamically adjust motor speeds based on error values, significantly improving accuracy, stability, and repeatability.

Additionally, advanced control methods such as adaptive control or fuzzy logic can be explored to enhance system performance in varying environmental conditions. These improvements can transform the system into a more robust and intelligent autonomous navigation platform.

IX. ENCODER SIGNAL PROCESSING

Encoders play a crucial role in the proposed system by providing real-time feedback on motor rotation and enabling accurate motion tracking. The encoders used in this system generate digital pulses corresponding to the rotation of the motor shaft. These pulses are directly related to the angular displacement of the wheels and serve as the primary source of feedback for navigation.

Each pulse generated by the encoder represents a small incremental movement of the wheel. By continuously counting these pulses, the system can estimate important motion parameters such as distance traveled, speed, and direction. The relationship between encoder pulses and wheel movement is determined by the number of pulses per revolution (PPR) and the radius of the wheel.

To ensure accurate pulse counting, interrupt-based techniques are employed. The encoder outputs are connected to the interrupt-capable GPIO pins of the ESP32 microcontroller. Whenever a pulse is detected, an interrupt service routine (ISR) is triggered, which increments the corresponding pulse counter. This method ensures that pulses are captured reliably without being affected by delays in the main program execution.

High-speed operation of motors can lead to rapid pulse generation, making it essential to use efficient and lightweight ISR routines. Care is taken to keep the interrupt handling

code minimal to avoid missing pulses and to maintain system responsiveness.

Noise and signal fluctuations are common issues in encoder systems, especially in electrically noisy environments. To address this, basic debouncing and signal filtering techniques are considered. Hardware-level pull-up resistors and proper grounding are used to stabilize the signal, while software techniques help in reducing false triggering.

The encoder data is processed in real time and stored in memory in a structured format. Each movement step is associated with corresponding encoder counts, allowing the system to reconstruct the path during autonomous operation. The stored data serves as a reference for reproducing the same trajectory in memory mode.

In addition to distance measurement, encoder signals can also be used to estimate velocity by calculating the rate of change of pulse counts over time. This information can be further utilized in advanced control strategies to improve motion accuracy.

Overall, the encoder signal processing mechanism forms the backbone of the feedback system, enabling precise tracking and reliable path reproduction in the proposed navigation robot.

X. WORKING PRINCIPLE

The system operates based on the interaction between user commands, control processing, and feedback mechanisms.

Initially, the user sends control commands through a Bluetooth-enabled device such as a smartphone. These commands are received by the ESP32 microcontroller via its Bluetooth module. The ESP32 processes the received instructions and determines the required motor actions.

Based on the command, the ESP32 sends appropriate control signals to the L298N motor driver. The motor driver then drives the motors in the desired direction and speed. For example, to move forward, both motors rotate in the same direction, whereas for turning, the motors rotate in opposite directions.

Simultaneously, the encoders attached to the motors generate pulses as the wheels rotate. These pulses are continuously monitored by the ESP32. By analyzing this feedback data, the system can calculate speed and displacement and make necessary adjustments to ensure accurate motion.

This closed-loop control mechanism improves system efficiency, reduces errors, and enhances overall performance.

XI. MATHEMATICAL MODEL

The distance traveled by the robot is calculated using encoder pulses:

$$D = N \times \frac{2\pi R}{PPR}$$

where:

- D = distance traveled
- N = number of pulses
- R = wheel radius

- PPR = pulses per revolution

The linear velocity of the robot is given by:

$$V = \frac{2\pi R \times N}{PPR \times t}$$

where:

- V = linear velocity
- t = time duration

For differential drive robots, the turning angle θ is given by:

$$\theta = \frac{D_R - D_L}{L}$$

where:

- D_R = distance traveled by right wheel
- D_L = distance traveled by left wheel
- L = distance between two wheels

Turning is achieved by varying wheel distances.

XII. METHODOLOGY

The proposed system operates in two distinct modes: manual mode and memory mode. These modes enable the robot to first learn a path through user interaction and then reproduce it autonomously using stored data. The methodology focuses on efficient path recording, structured data storage, and accurate path replay.

In **manual mode**, the robot is controlled using Bluetooth communication from a mobile device. The ESP32 microcontroller establishes a wireless connection and continuously listens for incoming commands such as forward, backward, left, right, and stop. Upon receiving a command, the microcontroller processes it and generates corresponding control signals, which are sent to the motor driver module.

The motor driver translates these control signals into appropriate voltage and current levels required to drive the motors. Depending on the command, the motors rotate in specific directions, enabling the robot to move accordingly. This phase allows the user to guide the robot along a desired path.

During manual operation, encoder pulses generated by the wheels are continuously monitored using interrupt-based techniques. These pulses are directly proportional to the rotation of the wheels and, consequently, the distance traveled. The encoder data is captured in real time and processed by the microcontroller.

To ensure efficient storage, the system records encoder values only when there is a change in movement direction. Each recorded step consists of the movement direction along with the corresponding encoder counts for both wheels. This structured data is stored sequentially in memory, forming a compact representation of the path.

Once the desired path has been completely recorded, the system transitions to **memory mode**. In this mode, the robot operates autonomously without requiring further user input. The stored path data is retrieved from memory and processed step-by-step.

For each stored step, the ESP32 generates corresponding motor control signals based on the recorded direction and

encoder values. The robot attempts to replicate the same movement by driving the motors accordingly. Encoder feedback is monitored during this phase to track progress, although real-time correction is limited.

The path reproduction process continues sequentially until all stored steps are executed. The robot then stops automatically upon completion of the path.

This methodology eliminates the need for complex navigation systems such as GPS, vision-based systems, or SLAM algorithms. It provides a simple and cost-effective solution for indoor navigation, particularly in environments where the path is repetitive and predefined.

Furthermore, the approach ensures low computational complexity, making it suitable for real-time embedded systems. The combination of manual teaching and autonomous execution enhances usability and flexibility, making the system practical for various applications.

Overall, the methodology emphasizes simplicity, efficiency, and reliability in achieving memory-based navigation using encoder feedback.

XIII. MEMORY MANAGEMENT

Memory management plays a critical role in the proposed system, as it directly affects the robot's ability to accurately record and reproduce navigation paths. The system employs a structured memory storage mechanism to efficiently store movement data in a sequential and organized manner.

Each movement of the robot is stored as a discrete step, consisting of two key parameters: the direction of motion and the corresponding encoder counts for both wheels. This structured representation allows the system to capture both the qualitative (direction) and quantitative (distance) aspects of motion.

An array-based data structure is used to store the sequence of steps. In the current implementation, a fixed-size array is allocated to store up to 200 steps. Each element in the array is defined using a structure that contains the movement direction along with left and right encoder counts. This approach ensures efficient access and retrieval of stored data during path replay.

During manual operation, encoder values are continuously monitored and stored only when there is a change in movement direction. This reduces redundant data storage and optimizes memory usage. By recording only significant transitions, the system minimizes unnecessary memory consumption while preserving the essential characteristics of the path.

The memory storage process follows a sequential approach, where each new step is appended to the array. A step index variable is maintained to track the current position in memory. This index is incremented with each stored step and later used to retrieve the path in the correct order during replay.

To ensure efficient memory utilization, only essential parameters are stored, avoiding duplication of data. This lightweight storage strategy is particularly important in embedded systems, where memory resources are limited.

During path reproduction, the stored data is accessed in sequence (or reverse sequence, depending on implementation) and used to generate corresponding motor control signals. This enables the robot to replicate the recorded path with minimal computational overhead.

One of the primary limitations of the current approach is the use of fixed memory allocation, which restricts the maximum length of the path that can be recorded. Once the array limit is reached, no additional steps can be stored. This constraint limits the scalability of the system for longer navigation tasks.

Additionally, since the memory is volatile, the stored path data is lost when the system is powered off. This limits the persistence of recorded paths across multiple sessions.

Future improvements can address these limitations by incorporating dynamic memory allocation techniques or integrating external storage solutions such as EEPROM, SD cards, or flash memory. These enhancements would allow storage of longer paths and enable persistent memory, making the system more flexible and practical for real-world applications.

Furthermore, data compression techniques can be explored to reduce memory usage, and indexing methods can be optimized to improve retrieval efficiency. Such advancements would significantly enhance the overall performance and scalability of the memory-based navigation system.

XIV. ALGORITHM

The overall operation of the proposed memory-based navigation robot is divided into two main phases: path recording (manual mode) and path reproduction (memory mode). The algorithm ensures systematic execution of these phases for accurate navigation.

- 1) Initialize the system by configuring the ESP32 microcontroller, motor driver, encoder inputs, and Bluetooth communication module.
- 2) Set all initial parameters, including encoder counters, memory storage index, and default motor states.
- 3) Establish Bluetooth connection with the external device (e.g., smartphone) for receiving control commands.
- 4) Continuously monitor for incoming Bluetooth commands.
- 5) Upon receiving a command (Forward, Backward, Left, Right, Stop), process the input and determine the corresponding motor action.
- 6) Execute motor control signals through the motor driver to move the robot in the desired direction.
- 7) Simultaneously, monitor encoder pulses using interrupt service routines to track wheel rotation in real time.
- 8) Record encoder counts along with the corresponding movement direction.
- 9) Store the recorded data as a structured step in memory only when a change in direction occurs, to optimize memory usage.
- 10) Repeat steps 4–9 until the complete path is recorded.
- 11) Upon receiving a specific command (e.g., replay command), switch the system from manual mode to memory mode.

- 12) Reset encoder counters and initialize path replay variables.
- 13) Retrieve stored path data sequentially from memory.
- 14) Generate corresponding motor control signals based on stored direction and encoder values.
- 15) Execute movements step-by-step to reproduce the recorded path.
- 16) Monitor execution until all stored steps are completed.
- 17) Stop the robot after completing the path.

This algorithm ensures efficient path recording and accurate reproduction, forming the core logic of the proposed navigation system.

XV. WORKING

The working of the proposed memory-based navigation robot is divided into two primary phases: manual operation (path recording) and autonomous operation (path reproduction). These phases enable the robot to first learn a path through user control and later execute the same path independently.

The operation begins in manual mode, where the user controls the robot using a Bluetooth-enabled mobile device. Commands such as forward, backward, left, right, and stop are transmitted wirelessly to the ESP32 microcontroller. The ESP32 receives these commands and processes them in real time, converting them into appropriate control signals.

These control signals are sent to the motor driver module, which drives the motors accordingly. Depending on the command, the motors rotate in specific directions, allowing the robot to move linearly or perform turning operations. This phase allows the user to guide the robot along a desired path.

While the robot is moving, the encoders attached to the motors continuously generate pulses corresponding to wheel rotation. These pulses are captured using interrupt-based techniques and counted in real time by the microcontroller. The encoder counts provide a precise measure of the distance traveled and the nature of motion.

Each movement command produces a distinct pattern of encoder counts, representing both direction and displacement. These values are processed and stored in memory in a structured format. To optimize memory usage, data is recorded primarily when there is a change in movement direction or command.

After the desired path has been completely recorded, the system transitions to memory mode. In this phase, the robot operates autonomously without requiring any external input. The stored encoder values and corresponding movement directions are retrieved from memory.

The ESP32 processes the stored data sequentially and generates motor control signals to replicate the recorded path. For each step, the robot executes the corresponding movement based on the stored encoder counts.

During path reproduction, encoder feedback is continuously monitored to track the robot's movement. The microcontroller compares the current encoder values with the stored reference values and ensures that each step is executed for

the intended distance. Although full real-time correction is not implemented, this comparison helps maintain reasonable accuracy.

The robot continues executing the stored sequence step-by-step until all recorded movements are completed. Once the entire path is reproduced, the system stops automatically.

This working mechanism enables autonomous navigation without relying on external systems such as GPS or vision-based sensors. It is particularly effective for repetitive tasks in structured indoor environments, where predefined paths need to be followed consistently.

Overall, the system demonstrates a simple, efficient, and reliable approach to memory-based navigation using encoder feedback and embedded control.

XVI. RESULTS AND DISCUSSION

The developed system was tested under controlled indoor conditions to evaluate its performance. The robot was manually driven along predefined paths, including straight lines, turns, and combined trajectories.

The encoder-based tracking system successfully recorded the path data and reproduced it during memory mode. The robot was able to follow the same path with reasonable accuracy.

The performance was evaluated based on the following factors:

- Accuracy of path reproduction
- Repeatability of movement
- Stability of motor control

It was observed that the system performs well for short distances. However, as the path length increases, small errors accumulate due to mechanical and environmental factors.

The main sources of error include:

- Wheel slippage
- Uneven surfaces
- Delay in motor response

Despite these limitations, the system provides a reliable solution for applications requiring repeated navigation.

XVII. PERFORMANCE METRICS

The performance of the robot is evaluated using the following metrics:

- **Accuracy:** Difference between recorded and reproduced path
- **Repeatability:** Ability to reproduce the same path multiple times
- **Response Time:** Delay between command input and execution
- **Stability:** Smoothness of robot motion
- **Energy Efficiency:** Power consumption during operation

These metrics help in analyzing the effectiveness of the system in real-world conditions.

Further experiments were conducted under different surface conditions such as smooth tiles, rough surfaces, and inclined planes.

It was observed that:

- Smooth surfaces provided highest accuracy
- Rough surfaces introduced minor deviations
- Inclined surfaces affected motor performance

The robot showed consistent repeatability under controlled conditions, making it suitable for structured environments.

XVIII. ERROR ANALYSIS

Error analysis is an important aspect of evaluating the performance of the system. Several factors contribute to inaccuracies in path reproduction.

Systematic Errors: These include calibration errors, unequal wheel diameters, and misalignment of components.

Random Errors: These arise due to environmental factors such as uneven surfaces and wheel slippage.

Cumulative Errors: Small errors accumulate over time, leading to larger deviations in long paths.

Mathematically, total error can be expressed as:

$$E_{total} = E_{systematic} + E_{random} + E_{cumulative}$$

Minimizing these errors is essential for improving system performance.

XIX. COMPARISON WITH EXISTING SYSTEMS

Various navigation techniques have been developed for mobile robots, each with its own advantages and limitations. The proposed memory-based navigation system is compared with commonly used methods such as GPS-based navigation, vision-based systems, and SLAM (Simultaneous Localization and Mapping).

GPS-based navigation systems are widely used for outdoor applications due to their simplicity and global coverage. However, they are not suitable for indoor environments because GPS signals are weak or unavailable inside buildings. This makes them ineffective for indoor robotic navigation.

Vision-based navigation systems use cameras and image processing techniques to detect and interpret the environment. While these systems can operate indoors, they require high computational power, complex algorithms, and are sensitive to lighting conditions. This increases both the cost and complexity of implementation.

SLAM-based systems provide advanced navigation capabilities by simultaneously mapping the environment and localizing the robot within it. Although highly accurate and versatile, SLAM systems require expensive sensors such as LiDAR and involve complex computations, making them less suitable for low-cost applications.

In contrast, the proposed memory-based navigation system offers a simple and cost-effective alternative. It does not rely on external sensors or complex algorithms. Instead, it uses encoder feedback to record and reproduce predefined paths, making it highly suitable for indoor environments where repetitive navigation is required.

The proposed system has low implementation cost and minimal computational requirements, making it ideal for educational purposes and small-scale automation tasks. However, unlike SLAM or vision-based systems, it lacks adaptability to dynamic environments and cannot perform real-time obstacle avoidance.

Overall, the comparison highlights that while advanced navigation systems offer higher flexibility and intelligence, the proposed system provides a practical and efficient solution for applications where simplicity, cost, and repeatability are the primary concerns.

XX. ADVANTAGES

The proposed memory-based navigation system offers several advantages that make it suitable for practical indoor applications.

- **Low Cost:** The system uses readily available and inexpensive components such as ESP32, DC motors, and encoders. This makes it affordable compared to advanced navigation systems that require LiDAR or vision sensors.
- **Simple Implementation:** The design and working of the system are straightforward. It does not require complex algorithms or high computational resources, making it easy to develop and maintain.
- **Suitable for Indoor Use:** Since the system does not rely on GPS, it performs effectively in indoor environments where GPS signals are unavailable or unreliable.
- **No External Sensors Required:** The robot uses only encoder feedback for navigation. This reduces system complexity and eliminates the need for additional sensing hardware.
- **Efficient for Repetitive Tasks:** The system is ideal for applications where the same path needs to be followed repeatedly, such as in warehouses or industrial settings.
- **Low Power Consumption:** Due to the use of minimal hardware components and simple processing, the system consumes relatively less power, increasing battery efficiency.

These advantages make the system a practical and scalable solution for small-scale autonomous navigation applications.

XXI. LIMITATIONS

Although the proposed system provides a simple and cost-effective navigation solution, it has certain limitations that affect its performance in real-world scenarios.

- **Error Accumulation:** The system relies on encoder feedback for tracking movement. Over long distances, small errors in encoder readings accumulate, leading to noticeable deviation from the original path.
- **No Obstacle Detection:** The robot does not have any sensors to detect obstacles. As a result, it cannot adapt to dynamic environments and may collide with objects present in its path.
- **Limited Accuracy:** The accuracy of the system is affected by factors such as wheel slippage, uneven surfaces,

and variations in motor performance. These factors introduce errors in path reproduction.

- **Lack of Real-Time Correction:** The system operates in an open-loop manner during path replay. Without feedback correction mechanisms, it cannot adjust its motion dynamically.
- **Dependence on Surface Conditions:** The performance of the robot varies depending on the type of surface. Smooth and uniform surfaces provide better accuracy compared to rough or uneven surfaces.

These limitations highlight the need for additional sensing and control mechanisms to improve the reliability and robustness of the system.

XXII. APPLICATIONS

The proposed memory-based navigation robot has a wide range of applications, particularly in structured indoor environments where repetitive path execution is required. Due to its simplicity, low cost, and reliability, the system can be effectively utilized in the following domains:

- **Robotics and Automation Systems:** The system can be integrated into small-scale robotic platforms for performing repetitive tasks such as material transport, inspection, and monitoring. Its ability to record and reproduce paths makes it suitable for automation in controlled environments.
- **Smart Vehicles and Remote-Controlled Cars:** The robot can be adapted for use in smart vehicle prototypes and remote-controlled systems. It can learn predefined routes and execute them autonomously, which is useful in developing intelligent transportation models and demonstration systems.
- **Industrial Automation and Control Systems:** In industrial environments, the robot can be used for material handling tasks, such as moving components between different workstations. Since industrial processes often involve fixed routes, the memory-based navigation approach provides an efficient and cost-effective solution.
- **Educational and Research Projects:** The system serves as an excellent educational tool for students and researchers to understand concepts related to embedded systems, robotics, control systems, and sensor integration. It provides hands-on experience in implementing real-time control and feedback mechanisms.
- **IoT-Based Control Systems:** The robot can be integrated with IoT platforms to enable remote monitoring and control. Data collected from the system can be transmitted to cloud platforms for analysis, making it suitable for smart automation applications.
- **Warehouse and Logistics Automation:** The system can be used in warehouses to transport goods along predefined paths. Since the routes are repetitive, the robot can efficiently reduce manual labor and improve operational efficiency.
- **Healthcare and Indoor Delivery Systems:** The robot can be utilized in hospitals and healthcare facilities to

deliver medicines, reports, and equipment along fixed routes. This reduces human effort and ensures timely delivery.

- **Service Robotics:** In environments such as hotels, restaurants, and offices, the robot can perform routine tasks like delivering items or guiding users along predefined paths.

Overall, the versatility and simplicity of the proposed system make it suitable for a wide range of practical applications, especially in environments where cost, efficiency, and repeatability are important factors.

- **Proper Insulation of Electrical Components:** All electronic components are properly insulated to prevent short circuits and accidental electrical contact. Protective casings and insulation materials are used to safeguard sensitive components from environmental factors such as dust and moisture.
- **Avoiding Overloading of Motors:** The motors are operated within their rated voltage and current limits to prevent overheating and potential failure. Continuous monitoring of motor operation ensures that excessive load conditions are avoided, thereby increasing the lifespan of the motors.
- **Ensuring Stable Power Supply:** A stable and regulated power supply is maintained to ensure proper functioning of the system. Voltage fluctuations can cause unpredictable behavior or damage to components such as the microcontroller. Proper grounding and voltage regulation techniques are implemented to maintain consistent performance.
- **Emergency Stop Functionality via Bluetooth:** An emergency stop mechanism is implemented through Bluetooth communication, allowing the user to immediately halt the robot in case of unexpected behavior. This feature ensures quick response and enhances operational safety.
- **Secure Wiring and Connections:** All electrical connections are properly secured to avoid loose contacts, which may lead to signal interruption or short circuits. Organized wiring also improves reliability and ease of maintenance.
- **Thermal Safety:** Components such as motors and motor drivers may generate heat during prolonged operation. Adequate ventilation and heat dissipation measures are considered to prevent overheating and ensure safe operation.
- **Mechanical Stability:** The robot is designed with a stable chassis and properly aligned wheels to avoid imbalance during motion. Secure mounting of all components reduces the risk of mechanical failure.

These safety considerations help in ensuring reliable operation, minimizing risks, and enhancing the overall robustness of the proposed robotic system.

XXIII. FUTURE SCOPE

The current system can be further enhanced by implementing advanced control techniques to improve accuracy

and reliability. One important improvement is the use of closed-loop control methods such as PID control, which can continuously correct the robot's motion based on real-time feedback from encoders.

The integration of obstacle detection sensors such as ultrasonic or infrared sensors can enable the robot to operate safely in dynamic environments. This will allow the system to avoid collisions and adapt its path when unexpected obstacles are encountered.

Sensor fusion techniques combining encoder data with additional sensors such as IMU (Inertial Measurement Unit) can significantly improve navigation accuracy. This will help in reducing errors caused by wheel slippage and uneven surfaces.

Another possible enhancement is the implementation of multiple path storage and selection. This would allow the robot to store different routes and execute them based on user requirements.

Furthermore, the system can be integrated with a mobile application or IoT platform for better control, monitoring, and data visualization. This would make the system more user-friendly and suitable for real-world applications.

These improvements can transform the current prototype into a more robust, intelligent, and scalable autonomous navigation system.

XXIV. CONCLUSION

A memory-based navigation robot was successfully designed and implemented using encoder feedback and an ESP32 microcontroller. The system is capable of recording a path during manual operation and reproducing the same path autonomously without relying on external navigation systems such as GPS.

The experimental results demonstrate that the robot can effectively follow predefined paths with reasonable accuracy in indoor environments. The use of encoder feedback provides a simple yet efficient method for tracking movement and storing path information.

Although minor deviations were observed due to factors such as wheel slippage and surface irregularities, the overall performance of the system is satisfactory for short-distance navigation tasks. The project highlights the feasibility of implementing a low-cost autonomous navigation solution using basic hardware components.

This work serves as a strong foundation for further improvements, including enhanced control mechanisms and integration of additional sensors. With these advancements, the system can be extended to more complex and dynamic applications in real-world environments.

XXV. SOFTWARE IMPLEMENTATION

The robot is controlled using an ESP32 microcontroller programmed in the Arduino framework. The software handles Bluetooth communication, motor control, encoder feedback processing, and path memory storage. It also ensures real-time coordination between input commands and feedback signals for accurate and efficient navigation.

```

1 // ===== LIBRARY =====
2 #include "BluetoothSerial.h"
3 BluetoothSerial SerialBT;
4
5 // ===== MOTOR PINS =====
6 #define IN1 26
7 #define IN2 27
8 #define IN3 32
9 #define IN4 33
10 #define ENA 25
11 #define ENB 14
12
13 // ===== ENCODER PINS =====
14 #define LEFT_ENC_A 34
15 #define RIGHT_ENC_A 35
16
17 volatile long leftCount = 0;
18 volatile long rightCount = 0;
19
20 // ===== PATH STORAGE =====
21 struct Step {
22     char dir;
23     long leftC;
24     long rightC;
25 };
26
27 Step path[200];
28 int stepIndex = 0;
29 char currentCmd = 'S';
30
31 // ===== ENCODER INTERRUPTS =====
32 void IRAM_ATTR leftISR() { leftCount++; }
33 void IRAM_ATTR rightISR() { rightCount++; }
34
35 // ===== MOTOR CONTROL =====
36 void forward() {
37     digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
38     digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
39 }
40
41 void backward() {
42     digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);
43     digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH);
44 }
45
46 void leftTurn() {
47     digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);
48     digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
49 }
50
51 void rightTurn() {
52     digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
53     digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH);
54 }
55
56 // ===== SAVE PATH STEP =====
57 void saveStep(char dir, long l, long r) {
58     if (stepIndex < 200 && dir != 'S') {
59         path[stepIndex++] = {dir, l, r};
60     }
61 }
62
63 // ===== PATH REPLAY =====
64 void reversePath() {
65     for (int i = stepIndex - 1; i >= 0; i--) {
66         char d = path[i].dir;
67
68         if (d == 'F') backward();
69         else if (d == 'B') forward();
70         else if (d == 'L') rightTurn();
71         else if (d == 'R') leftTurn();
72
73         delay(500);
74     }
75 }
76
77 // ===== SETUP =====
78 void setup() {
79     SerialBT.begin("ESP32_ROBOT");
80
81     pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
82     pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT);
83
84     pinMode(LEFT_ENC_A, INPUT_PULLUP);
85     pinMode(RIGHT_ENC_A, INPUT_PULLUP);
86
87     attachInterrupt(digitalPinToInterrupt(LEFT_ENC_A),
88                     leftISR, RISING);
89     attachInterrupt(digitalPinToInterrupt(RIGHT_ENC_A),
90                     rightISR, RISING);
91 }
92
93 // ===== LOOP =====
94 void loop() {
95     if (SerialBT.available()) {
96         char cmd = SerialBT.read();
97
98         if (cmd != currentCmd) {
99             saveStep(currentCmd, leftCount, rightCount);
100
101             leftCount = 0;
102             rightCount = 0;
103
104             if (cmd == 'F') forward();
105             else if (cmd == 'B') backward();
106             else if (cmd == 'L') leftTurn();
107             else if (cmd == 'R') rightTurn();
108             else if (cmd == 'V') reversePath();
109
110             currentCmd = cmd;
111         }
112     }

```

Listing 1. Encoder-Based Path Memory and Control Code

The above code demonstrates the implementation of Bluetooth-based control, encoder data acquisition, and path memory storage. The robot records movement in manual mode and reproduces it in reverse using stored encoder data.

REFERENCES

- [1] Espressif Systems, "ESP32 Technical Reference Manual," 2023.
- [2] Arduino, "Arduino Documentation," [Online]. Available: <https://www.arduino.cc>
- [3] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, "Introduction to Autonomous Mobile Robots," MIT Press, 2011.
- [4] S. Thrun, W. Burgard, and D. Fox, "Probabilistic Robotics," MIT Press, 2005.
- [5] J. Borenstein, H. R. Everett, and L. Feng, "Where Am I? Sensors and Methods for Mobile Robot Positioning," University of Michigan, 1996.
- [6] M. H. Rashid, "Power Electronics: Circuits, Devices, and Applications," Pearson, 2013.
- [7] D. P. Bertsekas, "Dynamic Programming and Optimal Control," Athena Scientific, 2005.
- [8] K. Ogata, "Modern Control Engineering," Prentice Hall, 2010.
- [9] Research papers on encoder-based navigation systems, IEEE Xplore Digital Library.
- [10] Research articles on embedded systems and robotics, SpringerLink.